SETU Code Lab
Design Document

Diarmuid O'Neill

South East Technological University

17/03/2026

Table of Contents

Table of Figures 4

Introduction 5

Hosting 5

Architecture Diagram 6

Sequence Diagrams 8

- Solve a Problem 8
- Login 9
- Sign Up 9
- View Problems 10
- Create Problem / Test Cases 10
- Update Problem / Test Cases 11
- Create / Update Course 11
- View Results 12
- View Profile 12
- View Leaderboard 13

Algorithms 14

- Test Harness for Running Submitted Code 14
- Approach to the use of Artificial Intelligence 15
- Automated Deployment Script 16
- Packages 17

Entity Relationship Diagram 18

- Design Notes 19

SQL Queries 20

- Login / Sign Up 20
- View Problems 20
- Solve Problem 20
- CRUD Problem 21
- CRUD Test Case 22
- CRUD Course 22
- View Results 24

View Profile 25
Leaderboard 25
Multi-Language Support 27
User Interface 28
Logo 28
High-Level UI Flow 29
Login 30
Sign Up 30
View Problems 31
View Problem > Solve Problem 31
View Problem > Solve Problem >Submission Alert 32
View Problem > Manage Problems 32
View Problem > Manage Problems > Create/Update Problem 33
View Problem > Manage Courses 33
View Problem > Manage Courses > Create/Update Course 34
View Problem > Manage Courses > View Course 34
View Problem > Manage Course > View Course > View Result 35
View Problem > Profile 35
View Problem > Leaderboard 36
Bibliography 37
Appendix 38

Table of Figures

Figure 1. Architecture Diagram 6
Figure 2. Solve a Problem sequence diagram 8
Figure 3. Login sequence diagram 9
Figure 4. Sign Up sequence diagram 9
Figure 5. View Problems sequence diagram 10
Figure 6. Create Problem / Test Cases sequence diagram 10
Figure 7. Update Problem / Test Cases sequence diagram 11
Figure 8. Create / Update Course sequence diagram 11
Figure 9. View Results sequence diagram 12
Figure 10. View Profile sequence diagram 12
Figure 11. View leaderboard sequence diagram 13
Figure 12. Test Harness code 14
Figure 13. Commands that run student code inside of docker container 14
Figure 14. AntiCheat.ts file to deter users from using generative AI 15
Figure 15. Entity Relationship diagram 18
Figure 16. Dynamic SQL statement, defines how leaderboard will be displayed 26
Figure 17. Logo 28
Figure 18. User Interface flow diagram 29
Figure 19. Login screen design 30
Figure 20. Sign Up screen design 30
Figure 21. View Problems screen design 31
Figure 22. Solve a Problem screen design 31
Figure 23. Submission alert design 32
Figure 24. Manage Problems screen design 32
Figure 25. Create / Update Problem screen design 33
Figure 26. Manage Courses screen design 33
Figure 27. Create / Update Course screen design 34
Figure 28. View Course screen design 34
Figure 29. View Result Screen design 35
Figure 30. View Profile screen design 35
Figure 31. Leaderboard screen design 36
Figure 32. Automated deployment script 16

Introduction

The purpose of this document is to outline the proposed design for SETU Code Lab. It will explore how each part of the project is intended to be implemented. It includes sections on hosting, sequence diagrams, important algorithms, important packages, database design and the user interface.

SETU Code Lab is an in-browser study tool intended to be used by students enrolled in computing related courses at SETU and their lecturers. Students can select and complete problems and view their results and statistics on their profile. Lecturers can create problems, and courses which allow them to assign problems to groups of students and easily track their progress. This will help students prepare for upcoming exams and coding interviews and will also help lecturers easily conduct and grade lab work. The platform will also use some gamification mechanics to keep students engaged.

Java has been selected as the first supported language for SETU Code Lab as it is the first language that new students are exposed to at SETU. Support for more languages such as Python or Go may be added in the future if time allows.

Hosting

The system will be hosted using DigitalOcean's Droplet service. This is a virtual private server (VPS). The $12 per month regular option provides 2GBs of RAM, 1 CPU, 50GB of SSD storage and 1TB of bandwidth which should be enough for SETU Code Lab with some optimization. The chosen operating system for this server is Ubuntu 24.04 as it is very stable, provides excellent Docker support and is familiar to the developer.

For security purposes an SSH key will be generated and used to access the server console. This is faster and more secure than the password option that is offered by DigitalOcean. The Github repository containing the project will be cloned onto the server and the needed dependencies will be installed such as Node.js, Node Package Manager (NPM) and Nginx for serving the frontend.

Architecture Diagram

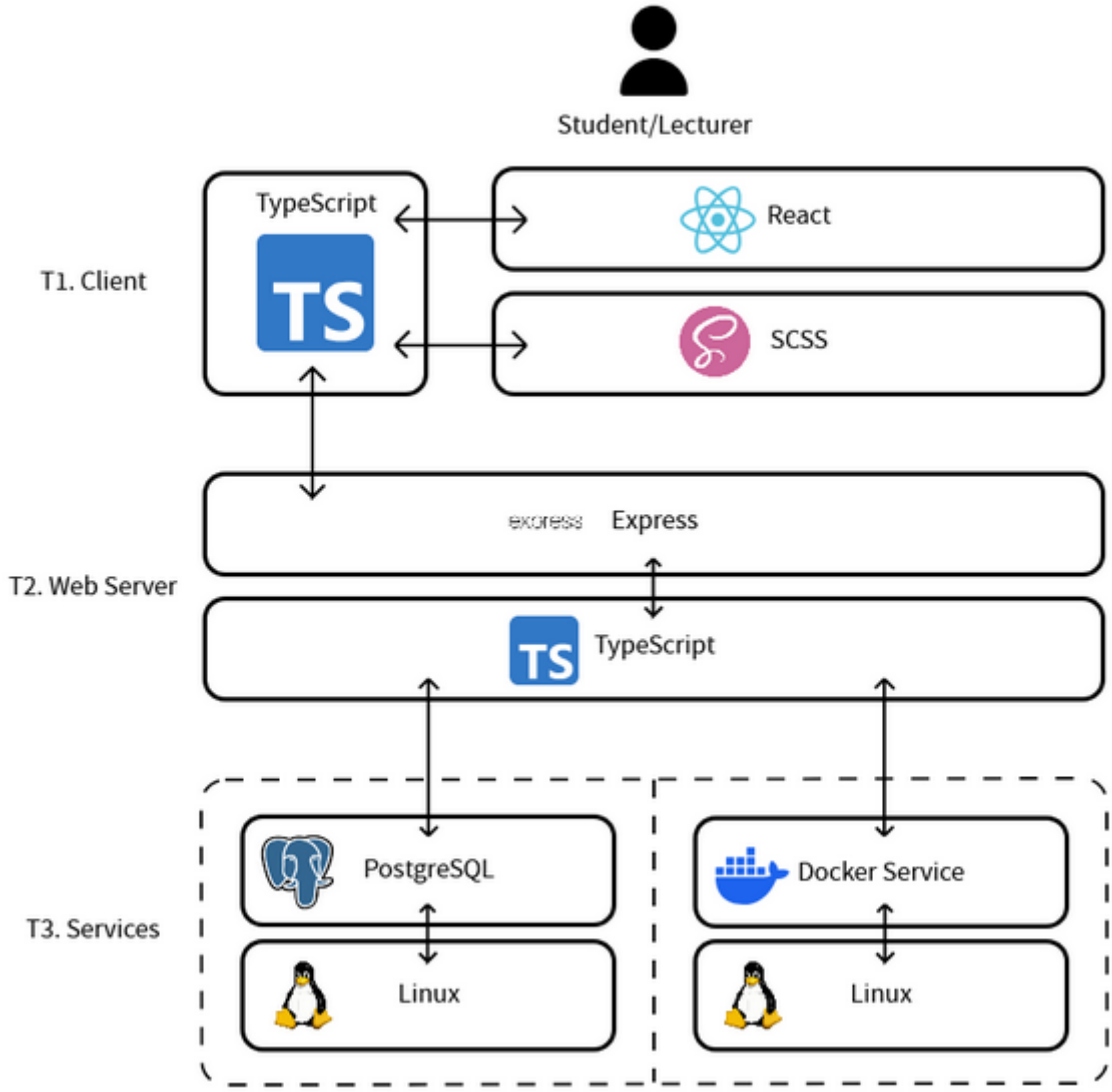

Figure 1. *Architecture Diagram*

SETU Code Lab uses a three-tier architecture. Tier one (T1) is the client and is built using React, Typescript and SCSS. This is where the user interface resides with everything required for each screen separated out into its own directory.

Tier two (T2) is the web server and is implemented using Node.js (this is a JavaScript runtime which allows TypeScript to run on a web server) and Express.js which is a Node.js framework used to simplify the routing process. The web server is structured into six layers: routes, middleware, controllers, services, models, and types. Routes define the API endpoints and direct incoming requests to the appropriate controller. Middleware handles cross-cutting concerns such as authentication and request parsing before requests reach the controller. Controllers process the incoming request and coordinate

the response. Services contain the core logic and interact with the models. Models handle direct communication with the PostgreSQL database. Types define the TypeScript interfaces and types used across the server.

Tier three (T3) is the services layer and consists of a PostgreSQL database which stores all the application data and the Docker service which is used to create temporary containers in which user submitted code runs in isolation.

Sequence Diagrams

Solve a Problem

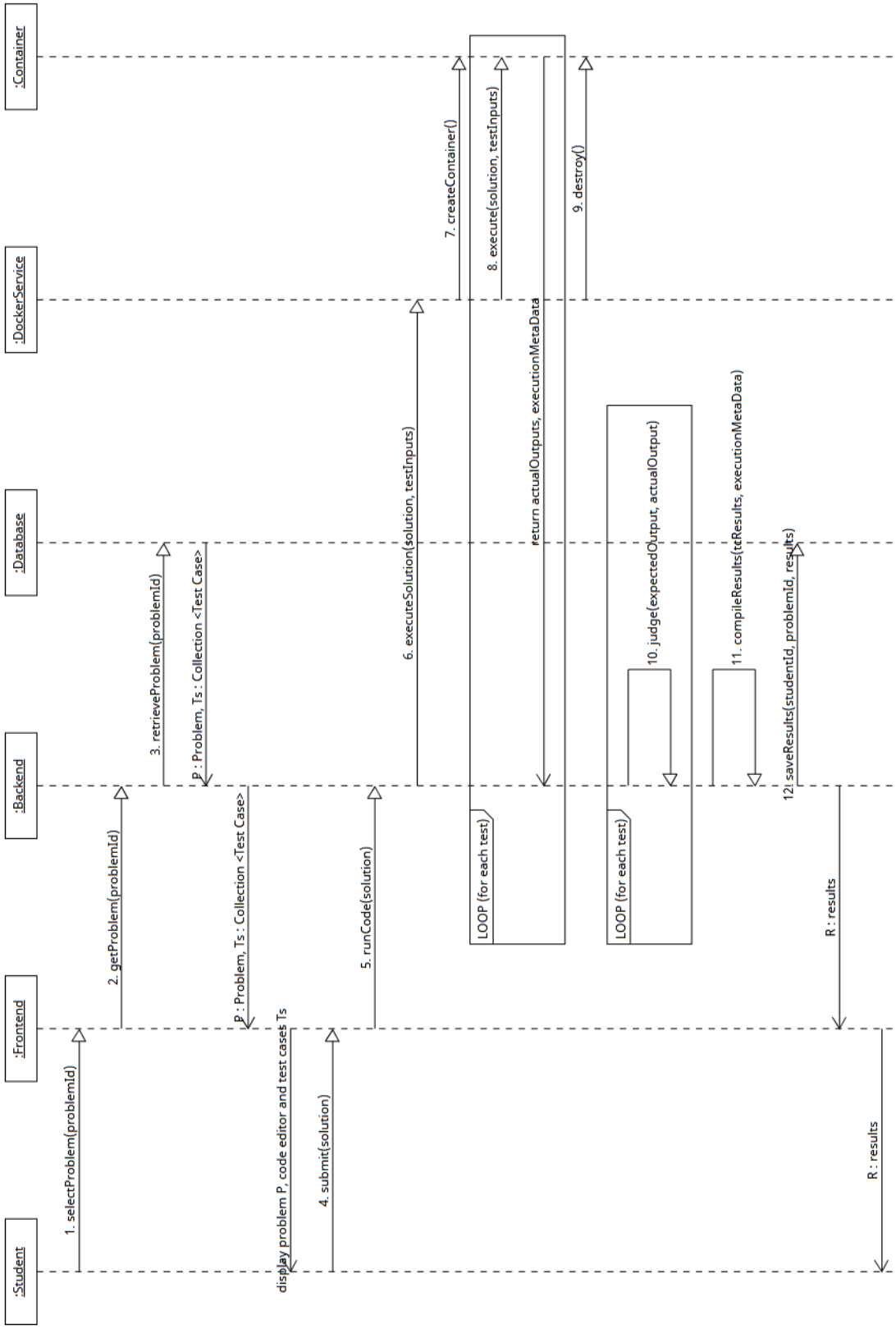

Figure 2. *Solve a Problem sequence diagram*

Login

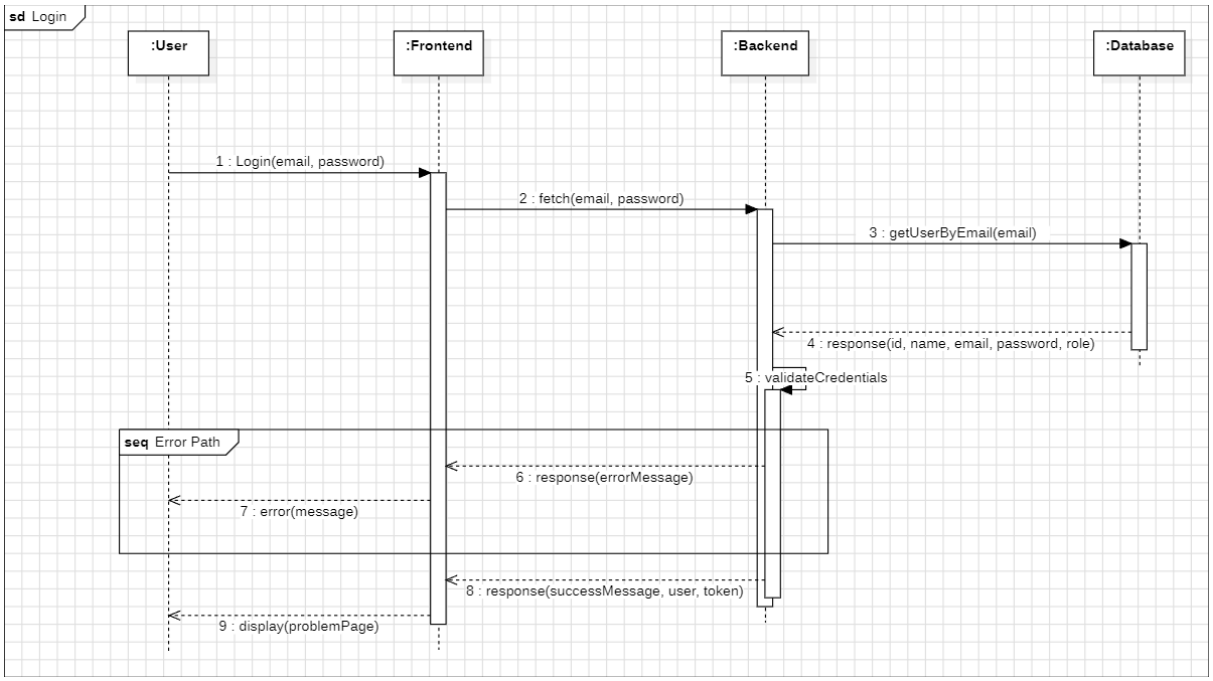

Figure 3. *Login sequence diagram*

Sign Up

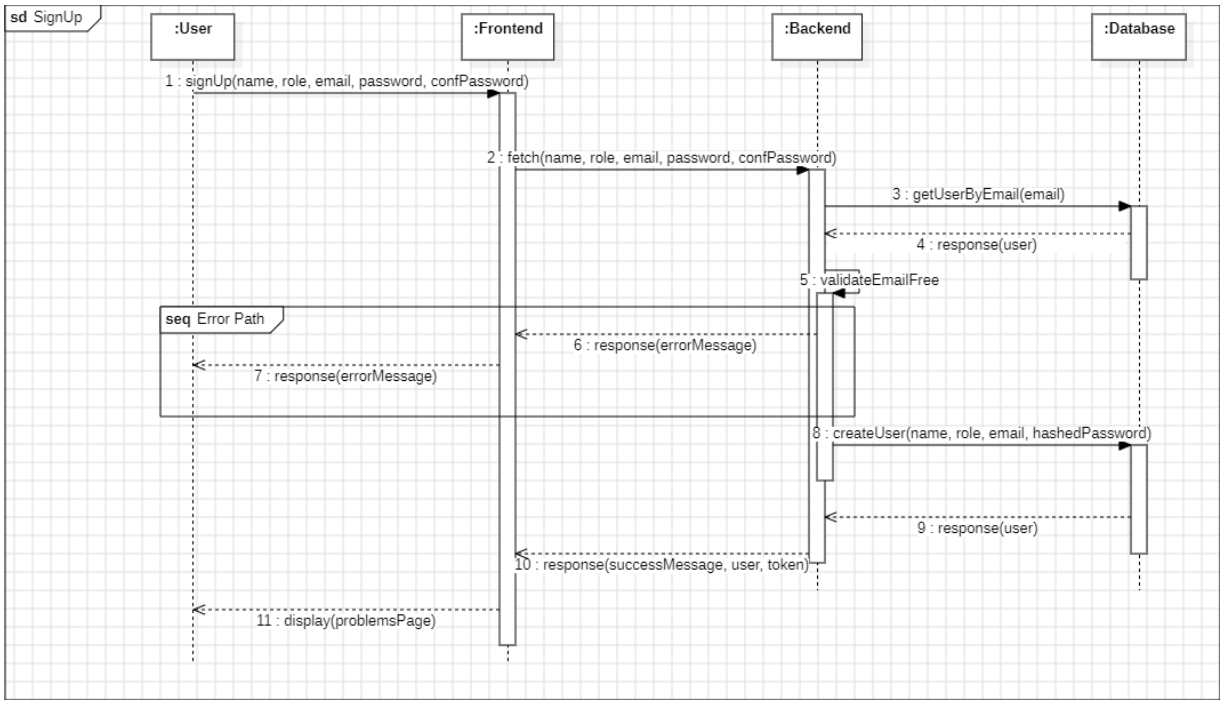

Figure 4. *Sign Up sequence diagram*

View Problems

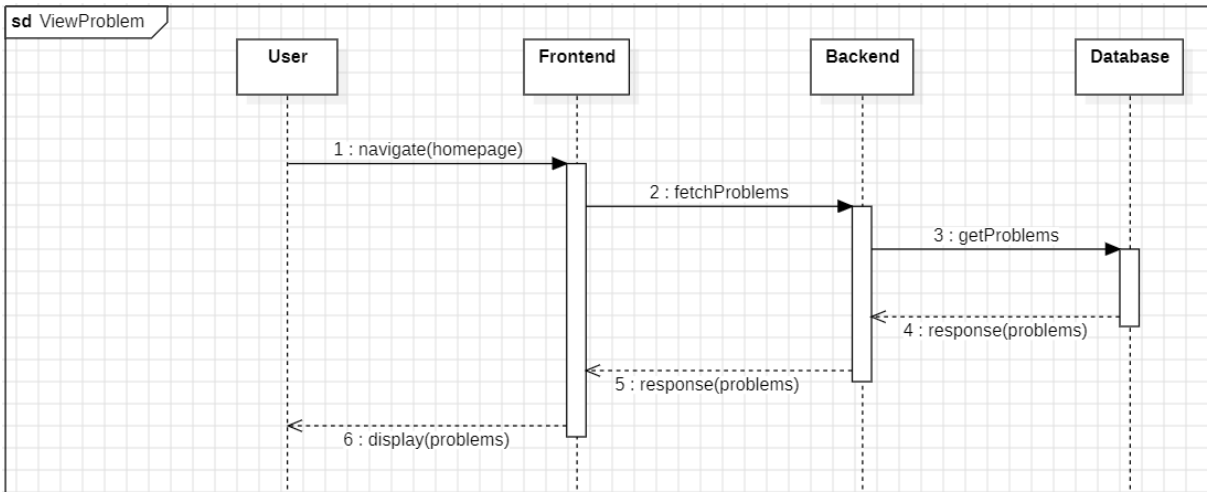

Figure 5. *View Problems sequence diagram*

Create Problem / Test Cases

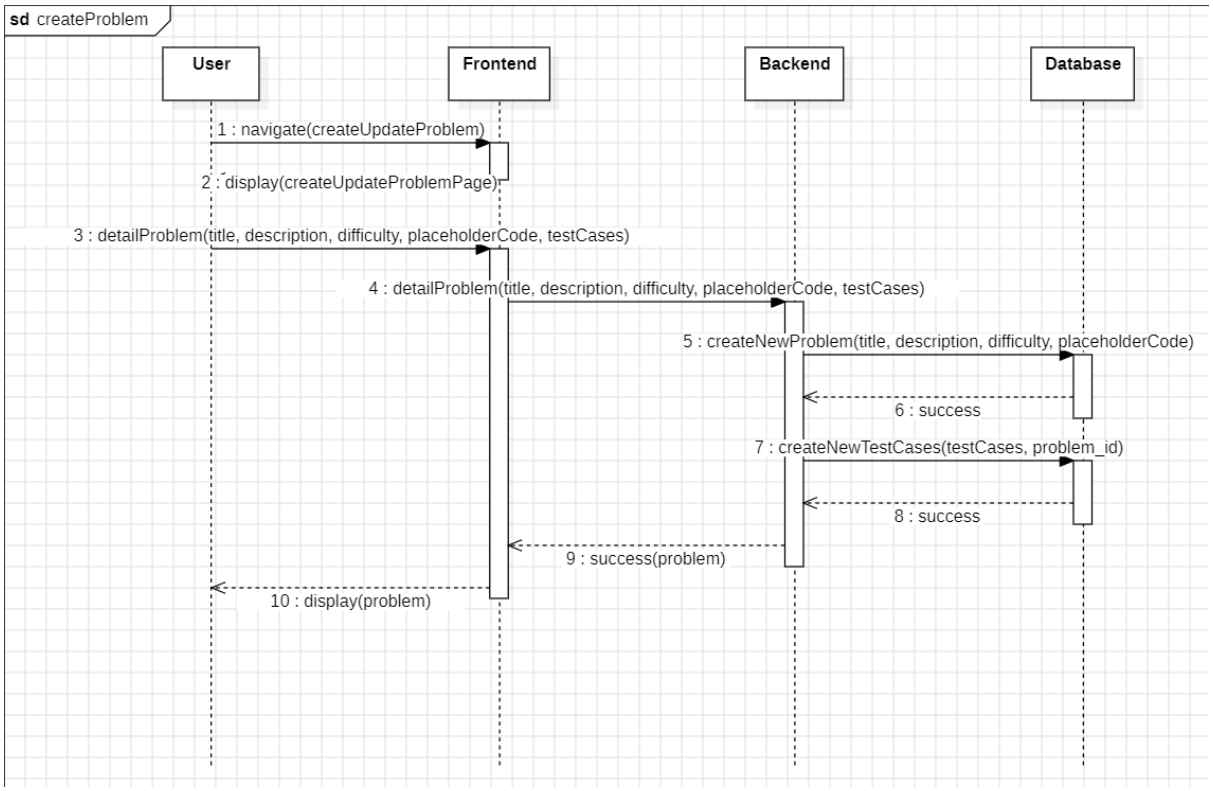

Figure 6. *Create Problem / Test Cases sequence diagram*

Update Problem / Test Cases

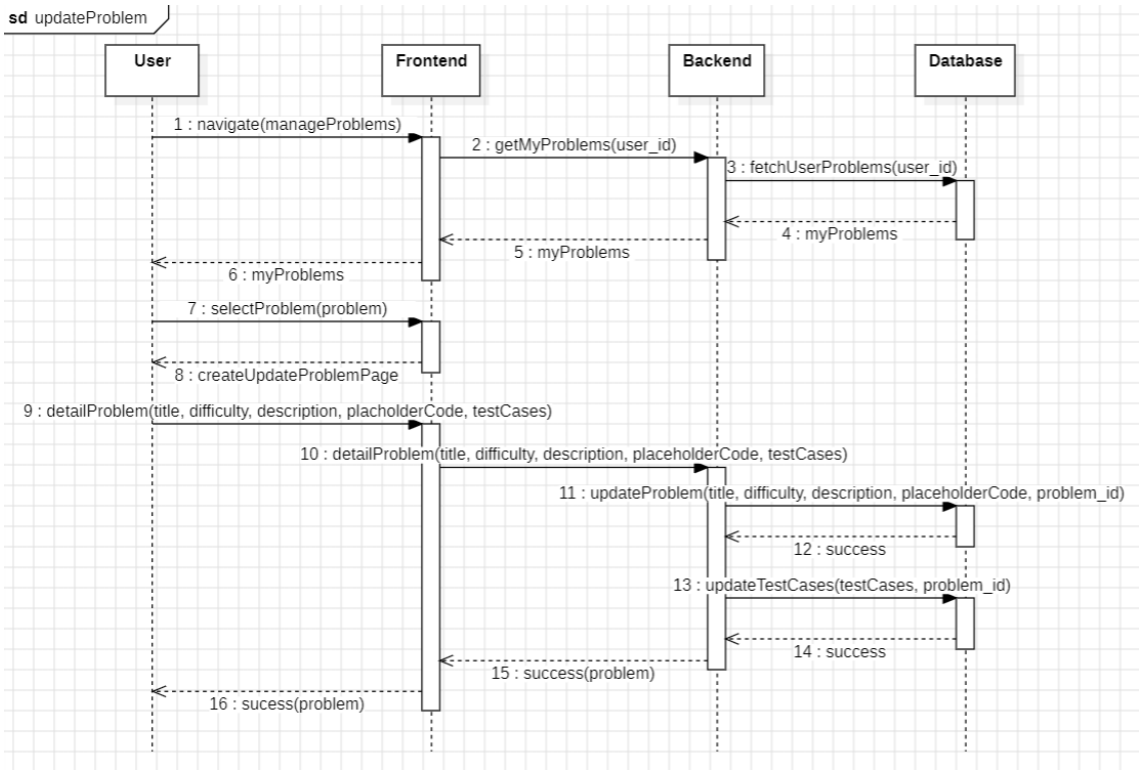

Figure 7. *Update Problem / Test Cases sequence diagram*

Create / Update Course

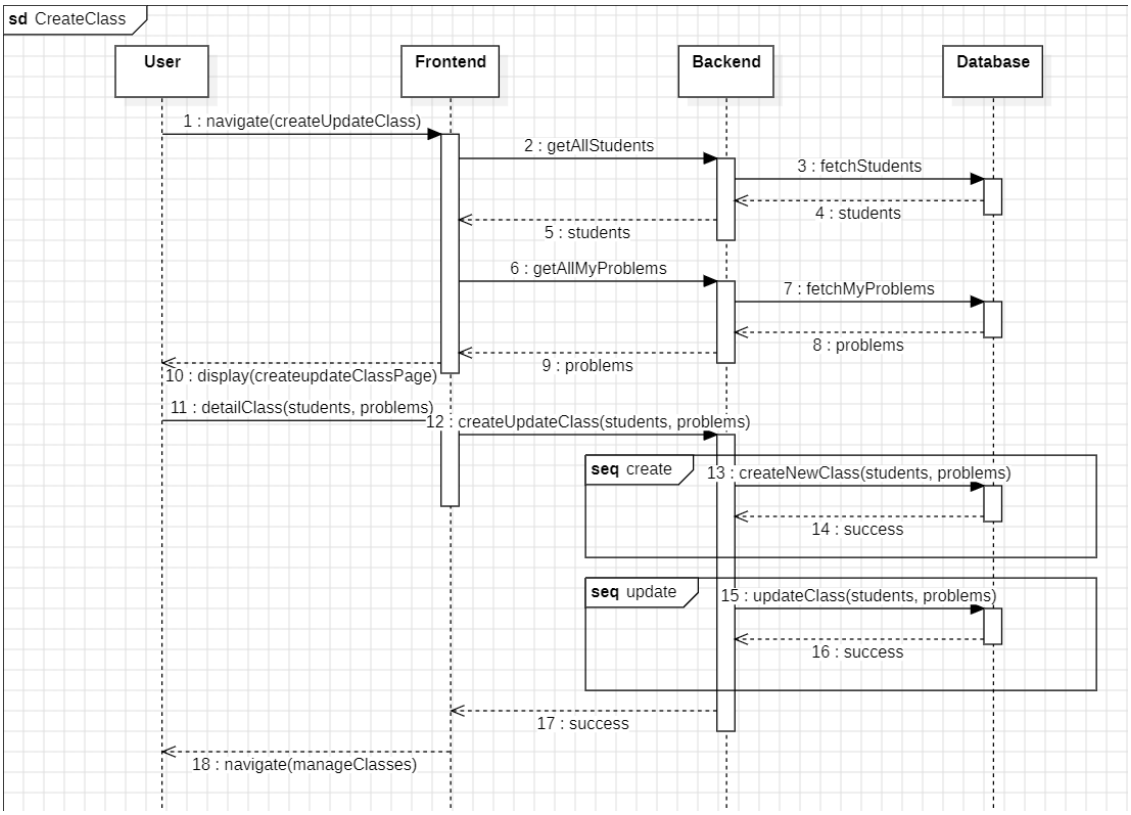

Figure 8. *Create / Update Course sequence diagram*

View Results

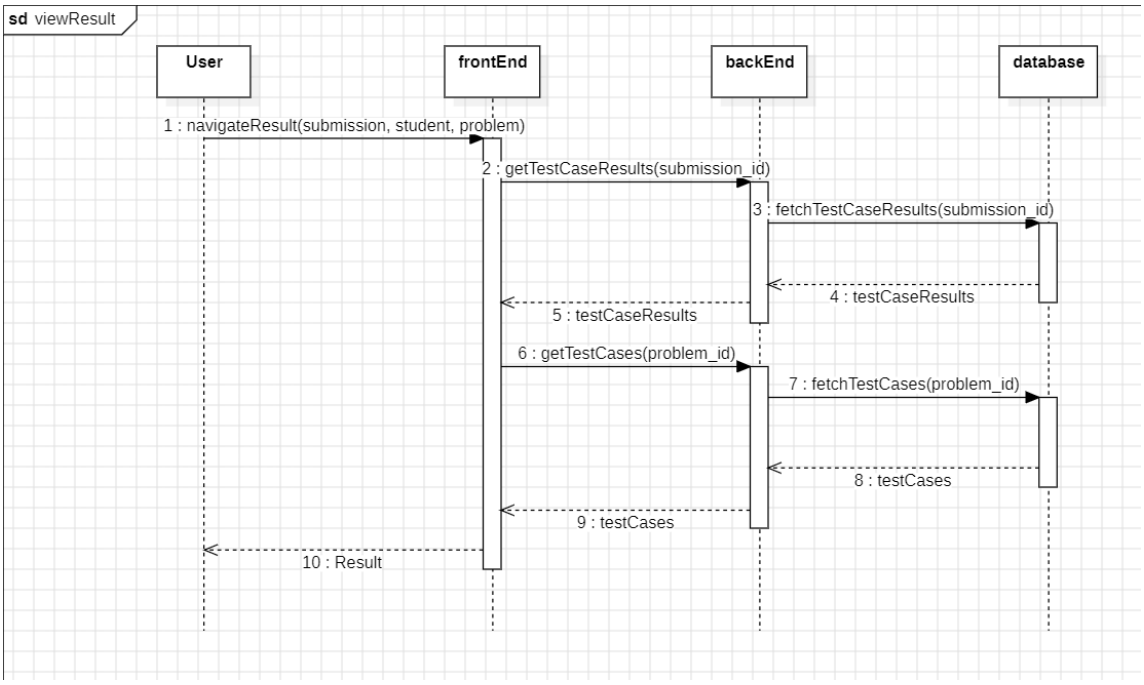

Figure 9. *View Results sequence diagram*

View Profile

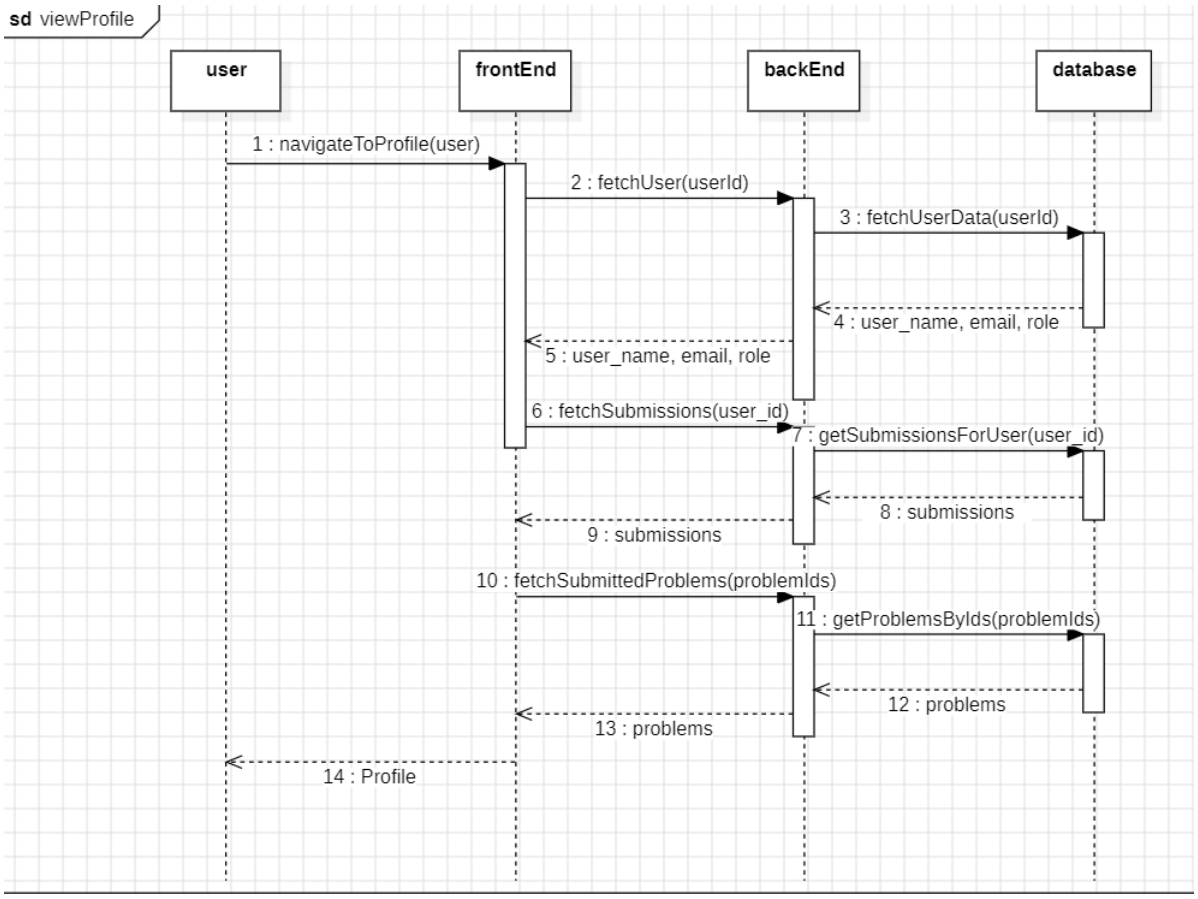

Figure 10. *View Profile sequence diagram*

View Leaderboard

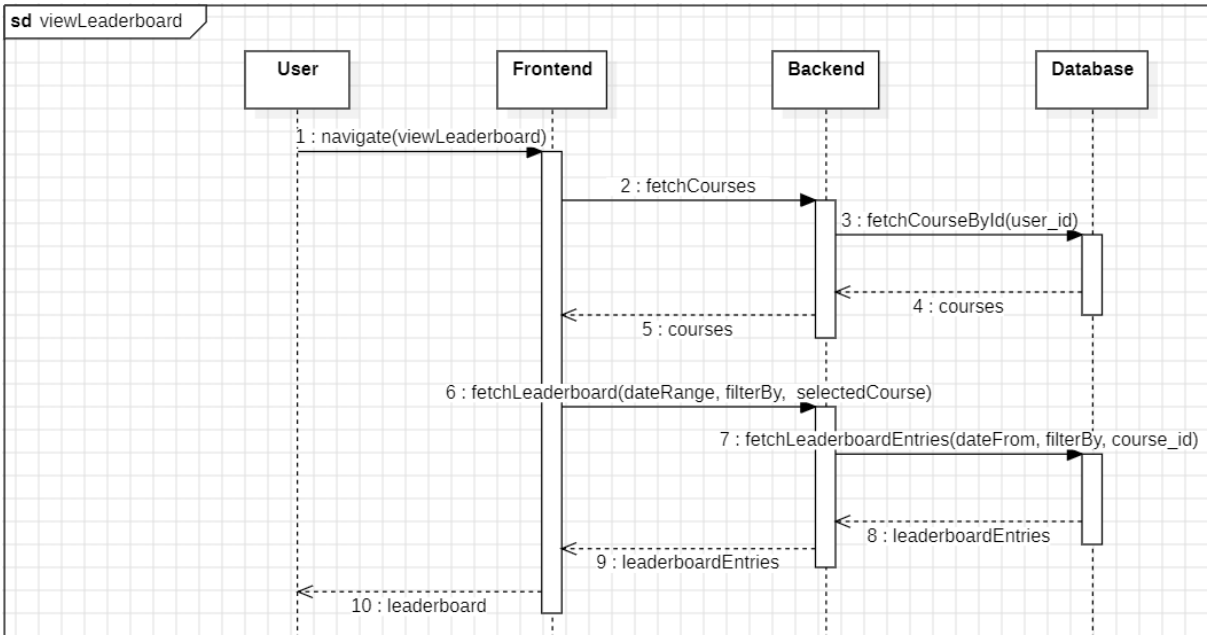

Figure 11. *View leaderboard sequence diagram*

Algorithms

Test Harness for Running Submitted Code

To run student submitted functions they must be injected into a static string harness which contains a main class and the needed dependency imports to map sample inputs to function parameters. This allows any inputted java function to run in isolation in a docker container with sample inputs from associated test cases. This harness is mostly the same for all submissions; however, some parts need to be dynamic to allow functions with parameters of different primitive and complex types and functions with different numbers of parameters to run. The harness looks like this:

```
const processedCode = `
import com.fasterxml.jackson.databind.ObjectMapper;
import java.util.*;

public class Main {
    static final ObjectMapper mapper = new ObjectMapper();
    ${code}
    static class Input {
        ${inputFields}
    }
    public static void main(String[] args) {
        try {
            Input input = mapper.readValue(args[0], Input.class);
            ${functionCallLine}
            System.out.println(mapper.writeValueAsString(result));
        } catch (Exception e) {
            System.out.println("ERROR:" + e.getMessage());
        }
    }
}
`;
```

Figure 12. *Test Harness code*

`${code}` is the student's submitted code, `${inputFields}` are all the input parameter types and names, and `${functionCallLine}` contains the final return type, the function itself and the input parameters. "`Input input = mapper.readValue(args[0], Input.class);`" maps sample inputs from problem test cases, these are passed as arguments when the main function is called. Below are the commands that run inside the docker container which allow the java code to execute with test case inputs (`${processedInput}` is the test case inputs).

```
        Cmd: ["sh", "-c", `
cat << 'EOF' > Main.java
${preprocessedCode}
EOF
javac Main.java
java -cp ".:/app/*" Main '${processedInput}'
```

Figure 13. *Commands that run student code inside of docker container*

Approach to the use of Artificial Intelligence

It is likely that some students using SETU Code Lab will attempt to use generative AI to assist them with completing problems. This is not how the platform is intended to be used as it hinders students' ability to learn coding concepts effectively. While it is impossible to prevent the use of generative AI entirely, the platform aims to increase the work factor to discourage most users from using it. A file called antiCheat.ts has been created which disables copy/paste functionality and can detect when a user switches tabs. This is only active on the "solve a problem" screen and can be bypassed in certain situations. This only acts as a deterrent to the use of AI assistance.

```
export const useAntiCheat = () => {
    const [shouldAutoSubmit, setShouldAutoSubmit] = useState(false);
    useEffect(() => {
        const prevent = (e: Event) => e.preventDefault();
        const preventContext = (e: MouseEvent) => e.preventDefault();
        const handleKeyDown = (e: KeyboardEvent) => {
            if ((e.ctrlKey || e.metaKey) && ["c", "v", "x", "a"].includes(e.key.toLowerCase())) {
                e.preventDefault()
                alert("Warning: Copy/Paste is disabled");
            }
        };

        const handleVisibility = () => {
            if (document.hidden) {
                setShouldAutoSubmit(true);
                alert("You left the tab. Your work has been automatically submitted.");
            }
        }
        document.addEventListener("visibilitychange", handleVisibility);
        document.addEventListener("copy", prevent);
        document.addEventListener("paste", prevent);
        document.addEventListener("cut", prevent);
        document.addEventListener("contextmenu", preventContext);
        document.addEventListener("keydown", handleKeyDown);
        return () => {
            document.removeEventListener("visibilitychange", handleVisibility);
            document.removeEventListener("copy", prevent);
            document.removeEventListener("paste", prevent);
            document.removeEventListener("cut", prevent);
            document.removeEventListener("contextmenu", preventContext);
            document.removeEventListener("keydown", handleKeyDown);
        };
    }, []);
    return { shouldAutoSubmit }
};
```

Figure 14. *AntiCheat.ts file to deter users from using generative AI*

Automated Deployment Script

The following script automates the deployment process. It works by pulling any code changes from the remote GitHub repository, running the frontend build step, running the backend build step and finally restarting the backend application.

```
#!/usr/bin/env bash

# Exit immediately if a command exits with a non-zero status
set -e

cd /var/www/SETU_Code_Lab || exit 1

git pull

cd /var/www/SETU_Code_Lab/SETU_Code_Lab_Code/client || exit 1

npm run build

cd /var/www/SETU_Code_Lab/SETU_Code_Lab_Code/server || exit 1

npm run build

pm2 restart setucl-backend

echo "Deployment Successful"
```

Figure 15. *Automated deployment script*

Packages

Several packages will be used during the development of SETU Code Lab. Some of these are necessary to build important features and some vastly improve user experience. These packages include:

- **react-timer-hook** – A custom react hook designed to simplify the implementation of timers and stopwatches. This is needed for the stopwatch that measures how long a student takes to complete a particular problem (Labib, 2025).
- **react-markdown** – Enables markdown support so that lecturers can neatly format problem descriptions (John Otander, 2025).
- **react-spinners** – Provides free, stylish, pre-built loading indicators for React applications (Hu, 2026).
- **dockerode** – Enables interaction between the web server and docker's remote API (Dias, 2025).
- **@types/dockerode** – Provides required dockerode types which are needed for use with TypeScript (TypeScript, 2026).
- **cookie-parser** – Middleware for Node.js and Express that enables easy parsing of cookies from incoming requests (S., 2025).
- **@codemirror/state @codemirror/view @codemirror/basic-setup –** This group of packages together provide the in-browser code editor that is needed to allow students to solve problems on SETU Code Lab (CodeMirror, 2026).
- **@codemirror/theme-one-dark** – This is a dark theme for the CodeMirror code editor and nicely fits the theming of the rest of SETU Code Lab.
- **@codemirror/lang-java** – Provides code editor support for Java such as language specific syntax highlighting (Haverbeke, 2025).
- **@codemirror/lang-python** – Provides code editor support for Python such as language specific syntax highlighting (Haverbeke, 2026).
- **@codemirror/language** – Implements required infrastructure to support different programming languages (Marijn Haverbeke, 2026).

Entity Relationship Diagram

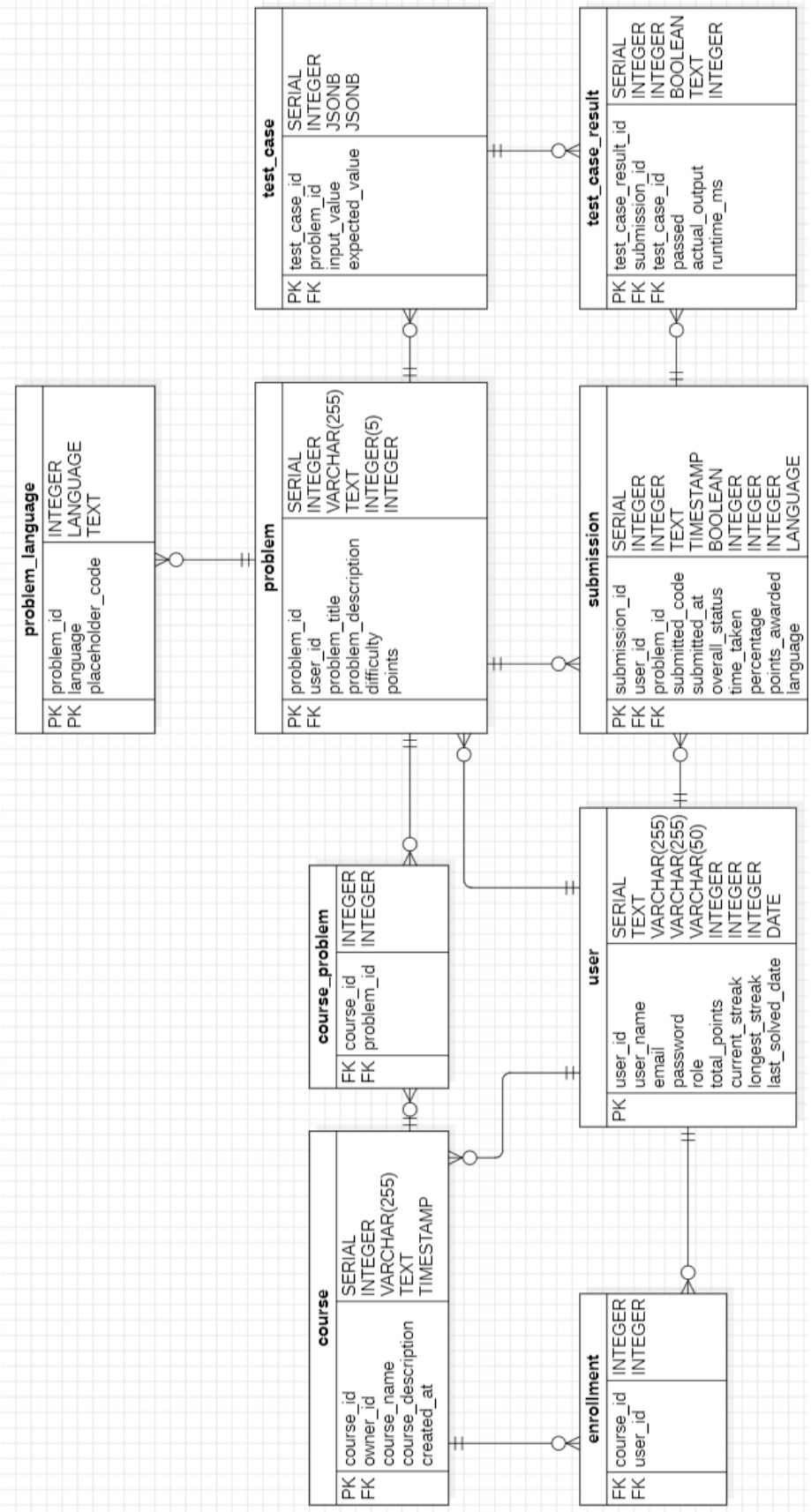

Figure 16. *Entity Relationship diagram*

Design Notes

- All tables use SERIAL primary keys for auto-incrementing IDs.
- Cascading deletes are applied on child tables (test_case, submission, course_problem, enrollment) so that removing a parent record automatically cleans up related data.
- The enrollment and course_problem tables use composite primary keys.
- problem.difficulty is constrained to values 1–5.
- users.role is constrained to 'student' or 'lecturer'.
- test_case.input_value and expected_value are stored as JSONB to support flexible, structured test data.
- Streak tracking (current_streak, longest_streak, last_solved_date) is maintained on the users table and updated at submission time.
- Exact SQL statements for table creation can be found in the Appendix

SQL Queries

Login / Sign Up

createUser

```
INSERT INTO users (user_name, role, email, password)
    VALUES ($1, $2, $3, $4)
    RETURNING *
```

getUserByEmail

```
SELECT user_id AS id, user_name AS name, email,
       password, role
    FROM users
    WHERE email = $1
```

View Problems

fetchProblems

```
SELECT problem.*, users.user_name
    FROM problem AS problem
    JOIN users AS users ON problem.user_id = users.user_id
```

Solve Problem

fetchTestCases

```
SELECT * FROM test_case WHERE problem_id = $1
```

createSubmission

```
INSERT INTO submission
    (user_id, problem_id, submitted_code, overall_status,
     time_taken, percentage, points_awarded)
    VALUES ($1, $2, $3, $4, $5, $6, $7)
    RETURNING *
```

createTestCaseResult

```
INSERT INTO test_case_result
    (submission_id, test_case_id, passed, actual_output, runtime_ms)
    VALUES ($1, $2, $3, $4, $5)
```

CRUD Problem

insertProblem

```
INSERT INTO problem
    (user_id, problem_title, problem_description,
     difficulty, points)
    VALUES ($1, $2, $3, $4, $5)
    RETURNING *
```

fetchProblemsByUserId

```
SELECT problem.*, users.user_name
    FROM problem AS problem
    JOIN users AS users ON problem.user_id = users.user_id
    WHERE problem.user_id = $1
```

getAllAvailableProblems

```
SELECT * FROM problem WHERE user_id = $1 OR user_id = 1
```

updateProblem

```
UPDATE problem
    SET problem_title=$1, problem_description=$2,
        difficulty=$3, points=$4
    WHERE problem_id = $5
    RETURNING *
```

deleteProblem

```
DELETE FROM problem WHERE problem_id = $1 RETURNING *
```

CRUD Test Case

createTestCase

```
INSERT INTO test_case (problem_id, input_value, expected_value)
    VALUES ($1, $2::json, $3::json)
    RETURNING *
```

updateTestCase

```
UPDATE test_case
    SET input_value=$1, expected_value=$2
    WHERE test_case_id = $3
    RETURNING *
```

deleteTestCase

```
DELETE FROM test_case WHERE test_case_id = $1 RETURNING *
```

CRUD Course

fetchAllStudents

```
SELECT user_id AS student_id, user_name AS student_name
    FROM users WHERE role = 'student'
```

insertCourse

```
INSERT INTO course (owner_id, course_title, course_description)
    VALUES ($1, $2, $3) RETURNING *
```

insertCourseProblem

```
INSERT INTO course_problem (course_id, problem_id)
    VALUES ($1, $2) RETURNING *
```

insertEnrollment

```
INSERT INTO enrollment (course_id, user_id)
    VALUES ($1, $2) RETURNING *
```

fetchCourseByUserId

```
SELECT c.* FROM course c
    JOIN enrollment e ON c.course_id = e.course_id
    WHERE e.user_id = $1 ORDER BY c.course_id ASC
```

fetchCreatedCourseByUserId

```
SELECT * FROM course WHERE owner_id = $1 ORDER BY course_id ASC
```

updateCourseDetails

```
UPDATE course SET course_title=$1, course_description=$2
    WHERE course_id = $3 RETURNING *
```

deleteCourseProblems

```
DELETE FROM course_problem WHERE course_id = $1
```

deleteEnrollmentsByCourseId

```
DELETE FROM enrollment WHERE course_id = $1
```

fetchProblemIdsFromCourse

```
SELECT problem_id FROM course_problem WHERE course_id = $1
```

fetchStudentIdsFromCourse

```
SELECT user_id FROM enrollment WHERE course_id = $1
```

View Results

fetchStudentsOnCourse

```
SELECT u.user_id, u.user_name FROM users u
    JOIN enrollment e ON u.user_id = e.user_id
    WHERE e.course_id = $1
```

getSubmissionsForCourse

```
SELECT DISTINCT ON (s.user_id, s.problem_id)
    s.submission_id, s.user_id, s.problem_id,
    s.submitted_code, s.submitted_at, s.overall_status,
    s.time_taken, s.percentage
    FROM submission s
    WHERE s.user_id = ANY($1)
      AND s.problem_id = ANY($2)
      AND s.submitted_at > $3::timestamp
    ORDER BY s.user_id, s.problem_id, s.submitted_at DESC
```

fetchTestCaseResults

```
SELECT * FROM test_case_result WHERE submission_id = $1
```

View Profile

fetchUserData

```
SELECT user_id, user_name, email, role, total_points,
       current_streak, longest_streak, last_solved_date
    FROM users WHERE user_id = $1
```

deleteAccount

```
DELETE FROM users WHERE user_id = $1
```

getSubmissionsForUser

```
SELECT * FROM submission
    WHERE user_id = $1 ORDER BY submitted_at DESC
```

getProblemsByIds

```
SELECT * FROM problem WHERE problem_id = ANY($1::int[])
```

Leaderboard

updateUserPoints

```
UPDATE users SET total_points = total_points + $1
    WHERE user_id = $2
```

getStreakData

```
SELECT last_solved_date, current_streak, longest_streak
    FROM users WHERE user_id = $1
```

updateStreak

```
UPDATE users
    SET current_streak = $1,
        longest_streak = GREATEST(longest_streak, $1),
        last_solved_date = CURRENT_DATE
    WHERE user_id = $2
```

checkFirstSolve

```
SELECT submission_id FROM submission
    WHERE user_id = $1 AND problem_id = $2 AND overall_status = true
```

fetchLeaderboardEntries

```
SELECT DISTINCT u.user_id, u.user_name, u.total_points,
    u.current_streak, u.longest_streak
    FROM users u
    INNER JOIN submission s ON u.user_id = s.user_id
    INNER JOIN course_problem cp ON s.problem_id = cp.problem_id
    WHERE cp.course_id = $1
```

`$1` defines how the data what data will be displayed in the leaderboard and in what way dynamically (see figure 16).

```
const values: any[] = [courseId];
let paramIndex = 2;

if (dateFrom) {
    query += ` AND u.role = 'student' AND s.submitted_at >= $${paramIndex}`;
    values.push(dateFrom);
    paramIndex++;
} else {
    query += ` AND u.role = 'student'`;
}

if (filterBy === "score") {
    query += ` ORDER BY u.total_points DESC`;
} else if (filterBy === "currentStreak") {
    query += ` ORDER BY u.current_streak DESC`;
} else {
    query += ` ORDER BY u.total_points DESC`;
}
```

Figure 17. *Dynamic SQL statement, defines how leaderboard will be displayed*

Multi-Language Support

getProblemLanguageData

```
SELECT * FROM problem_language
    WHERE problem_id = $1
```

insertProblemLanguage

```
INSERT INTO problem_language (problem_id, language, placeholder_code)
    VALUES problem_id = $1
```

deleteProblemLanguages

```
DELETE FROM problem_language
    WHERE problem_id = $1
```

User Interface

This section outlines the initial designs for every screen on the SETU Code Lab platform and how they flow from one screen to the next. It also includes other designs that may not necessarily be a screen for example the logo and the submission alert. Note that these designs are only for illustrative purposes and may not reflect exactly what the final platform looks like.

Logo

This is the logo and favicon for SETU Code Lab. It has been designed in the shape of the letter C and is intended to be simple and recognizable. The light grey and bright red colours have been chosen as they contrast well with the dark background chosen for the rest of the platform.

Figure 18. *Logo*

High-Level UI Flow

The diagram below shows how to navigate to each screen on the platform. Additional UI elements such as pop-ups and drop-down menus are not shown here, just the main screens. Items in the lecturer only box are only accessible to users with the lecturer role.

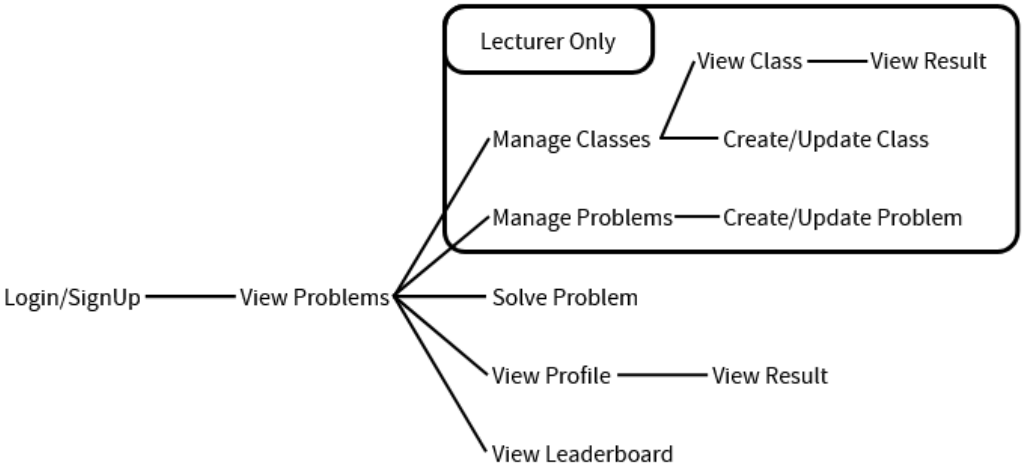

Figure 19. *User Interface flow diagram*

Login

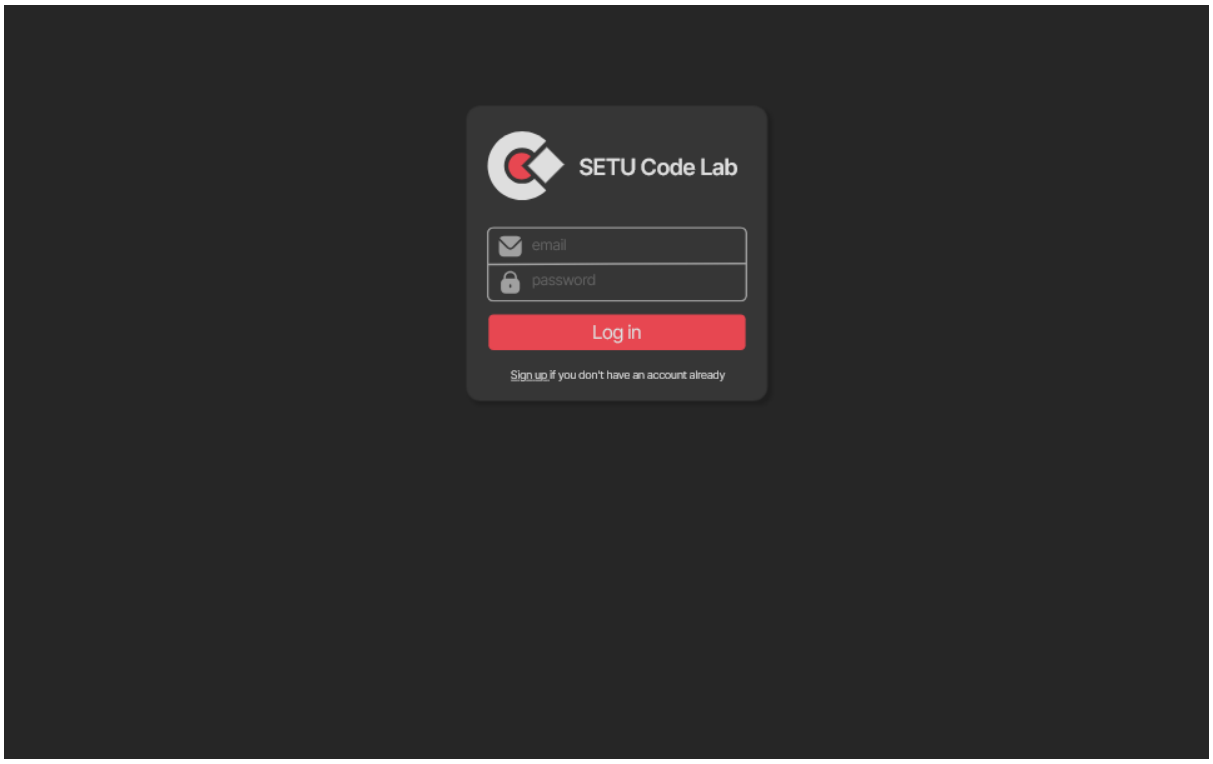

Figure 20. *Login screen design*

Sign Up

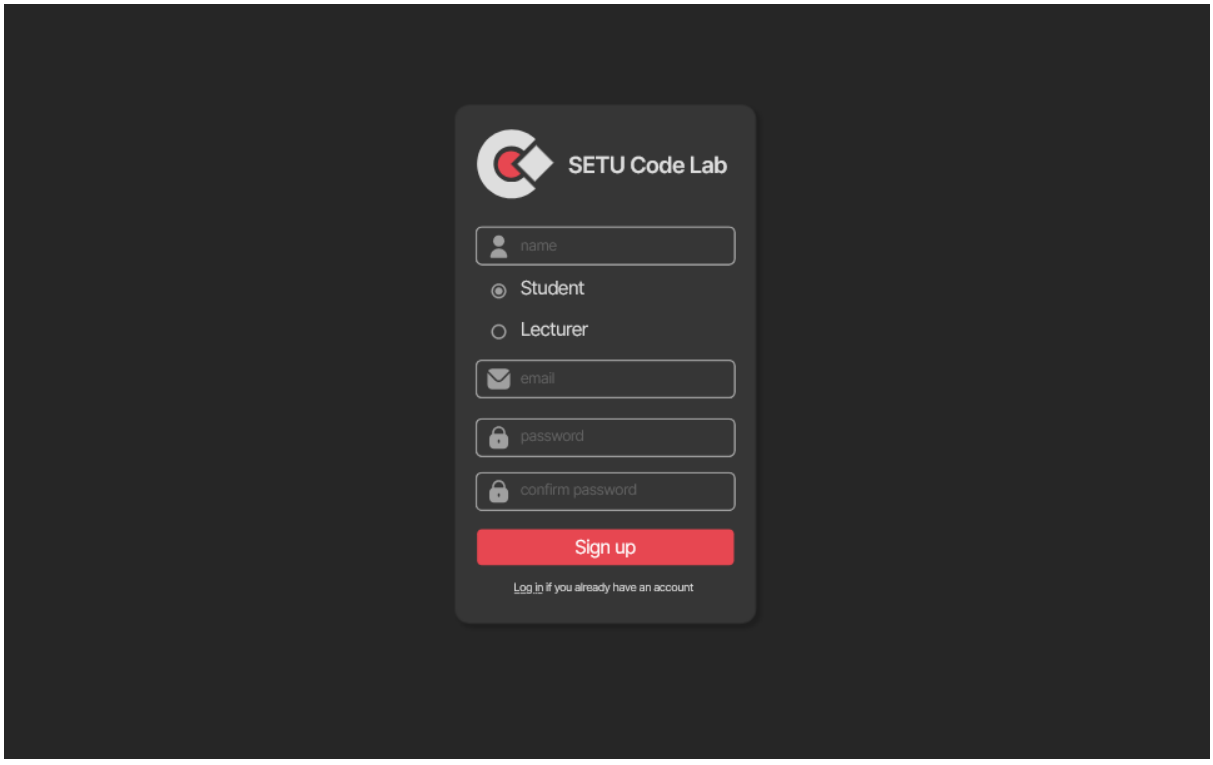

Figure 21. *Sign Up screen design*

View Problems

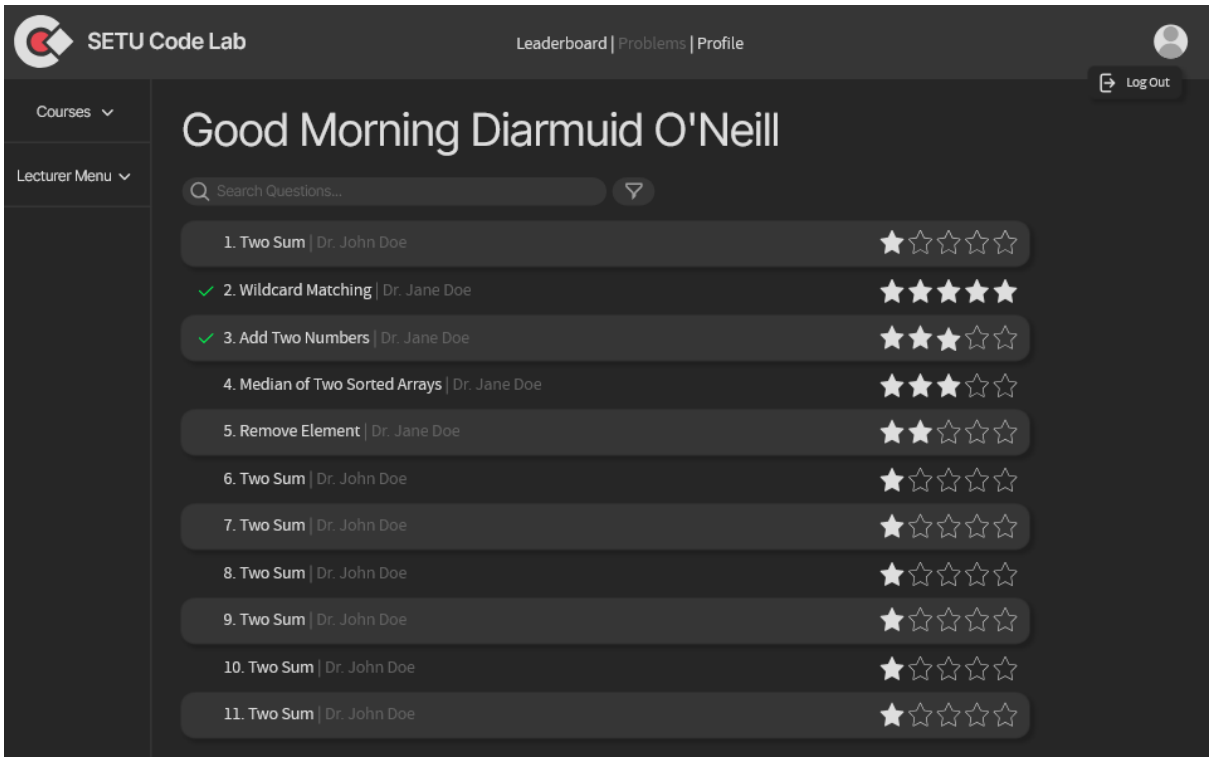

Figure 22. *View Problems screen design*

View Problem > Solve Problem

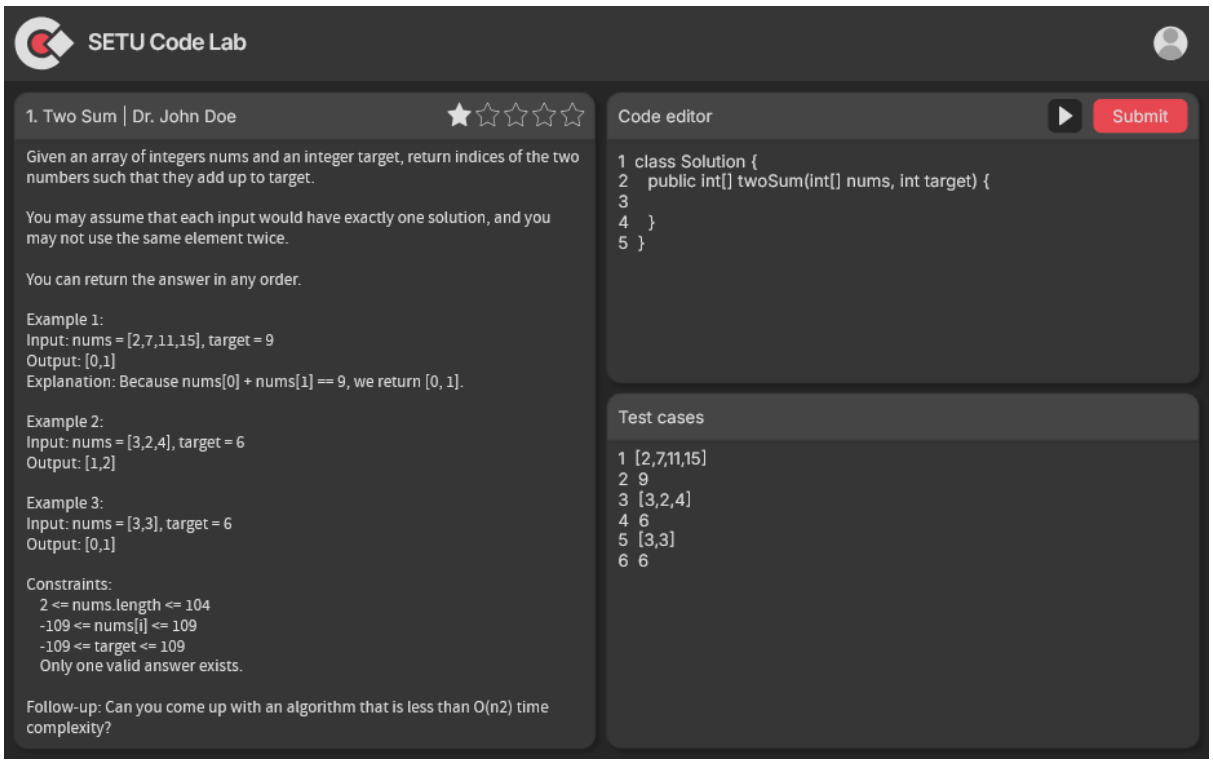

Figure 23. *Solve a Problem screen design*

View Problem > Solve Problem >Submission Alert

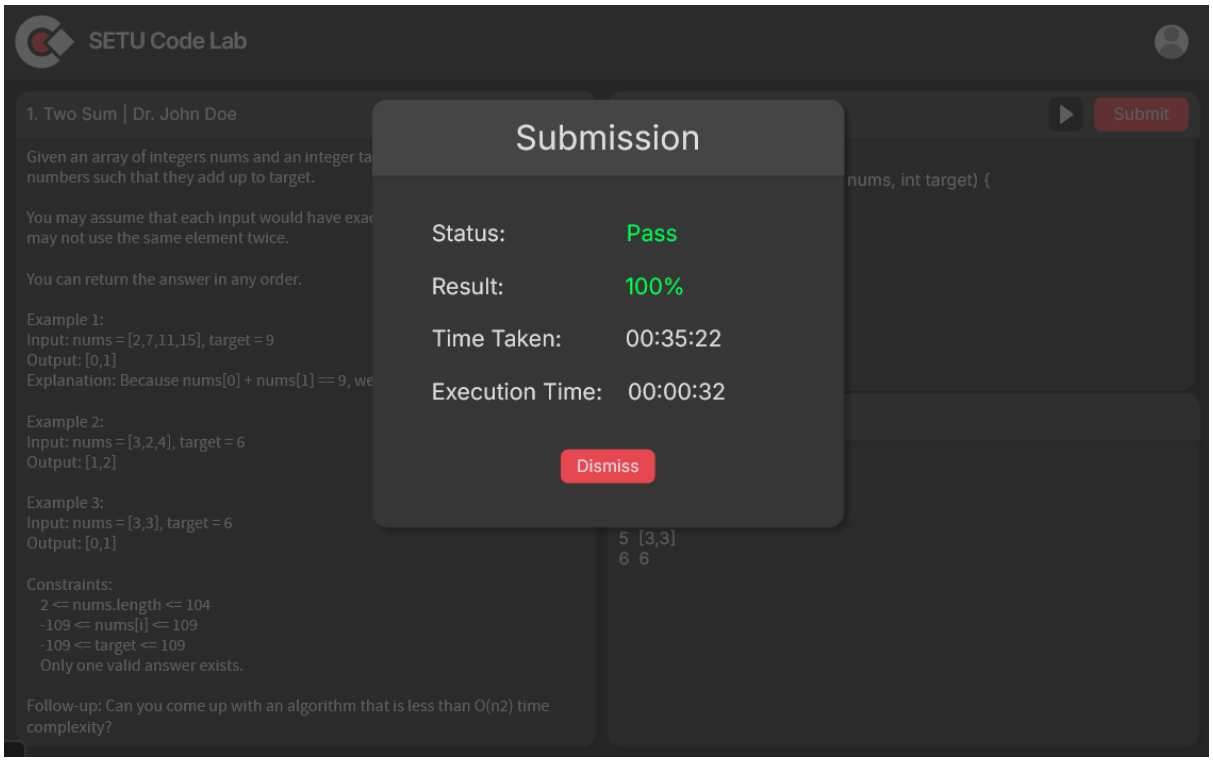

Figure 24. *Submission alert design*

View Problem > Manage Problems

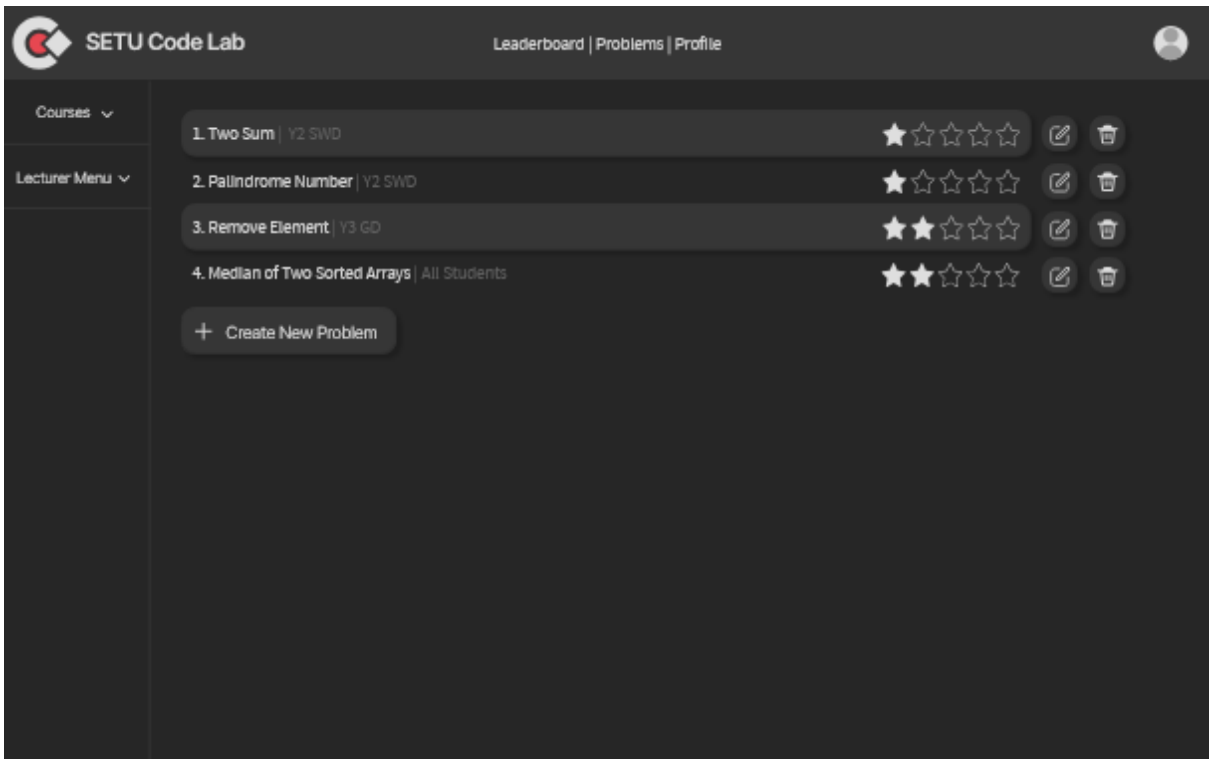

Figure 25. *Manage Problems screen design*

View Problem > Manage Problems > Create/Update Problem

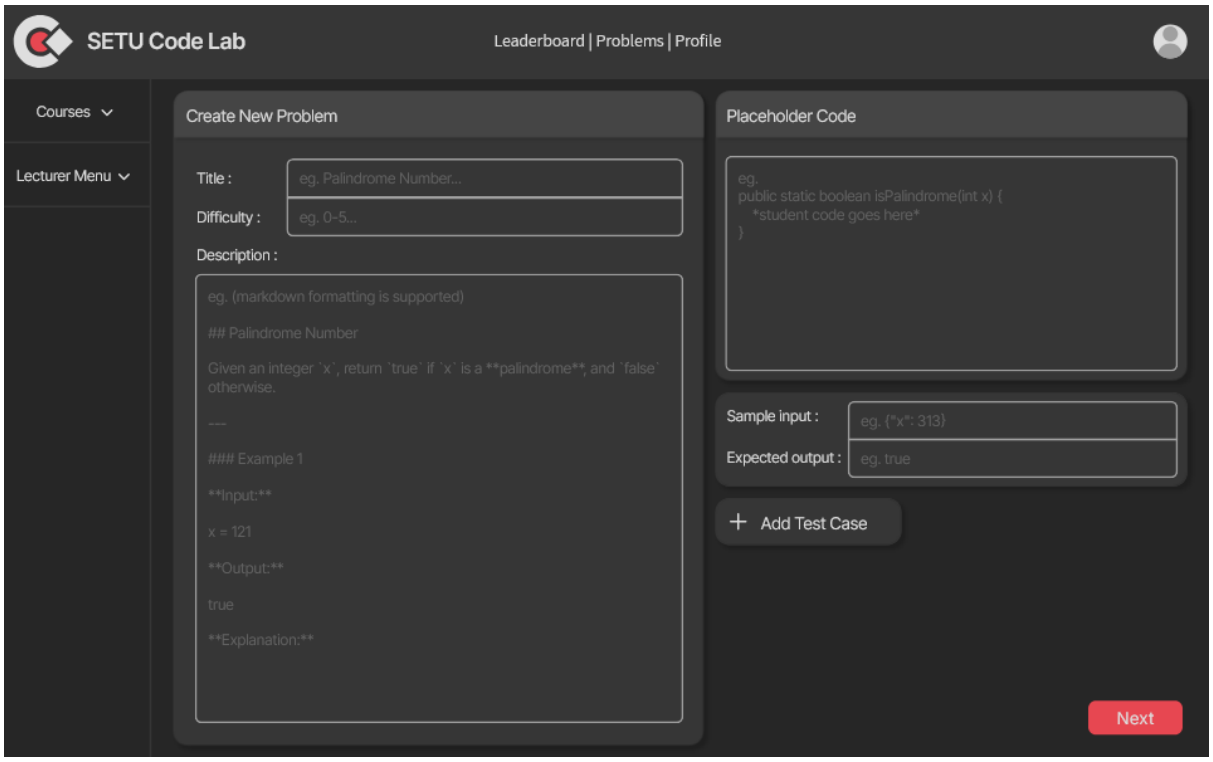

Figure 26. *Create / Update Problem screen design*

View Problem > Manage Courses

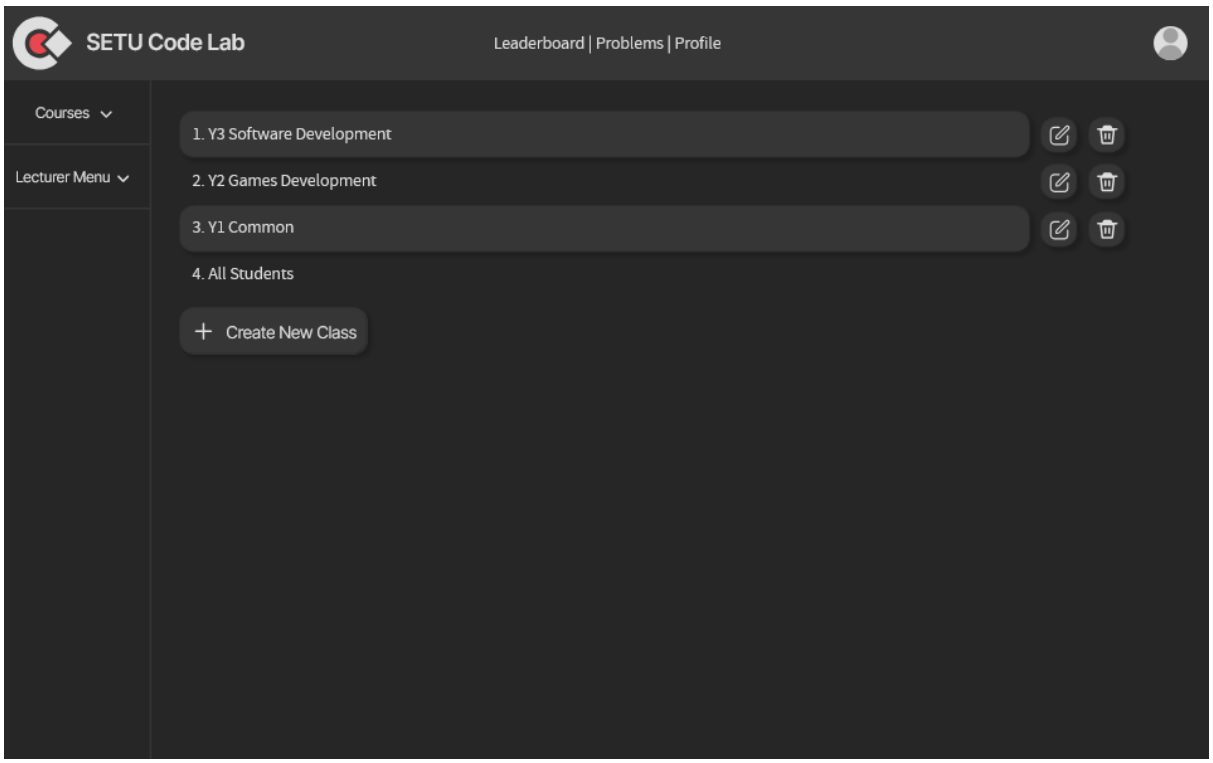

Figure 27. *Manage Courses screen design*

View Problem > Manage Courses > Create/Update Course

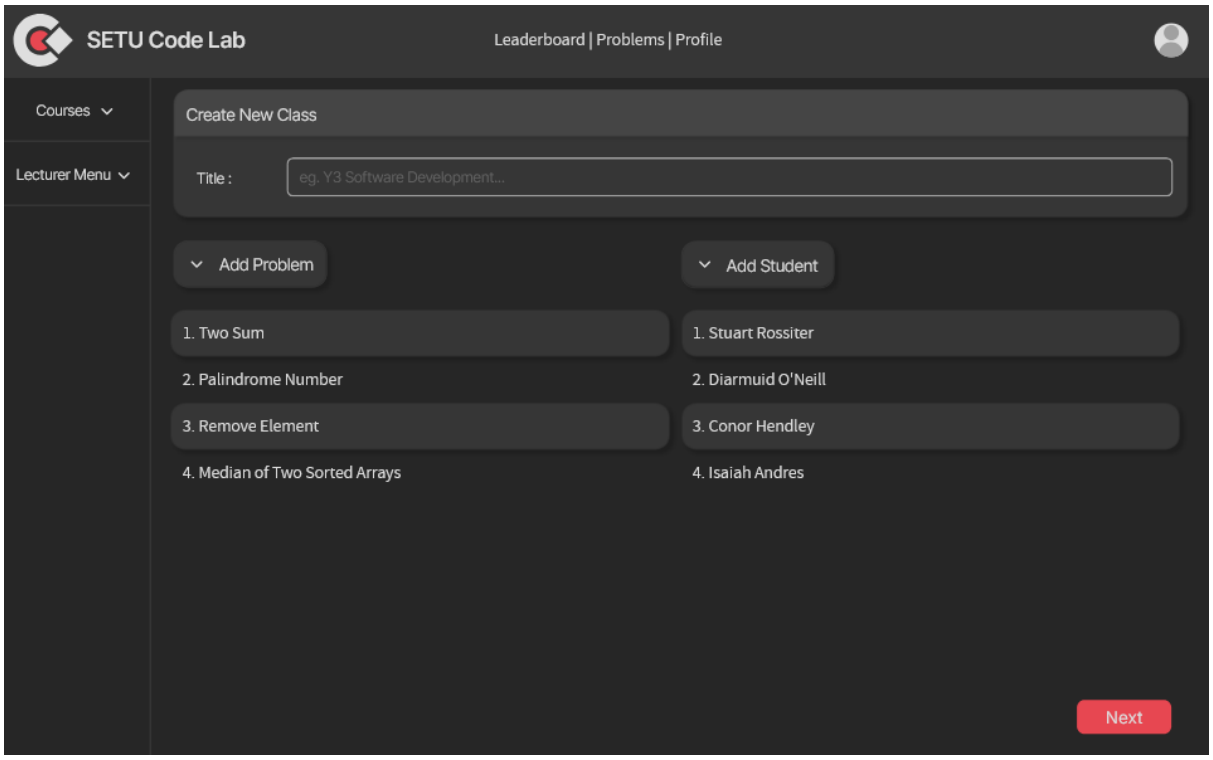

Figure 28. *Create / Update Course screen design*

View Problem > Manage Courses > View Course

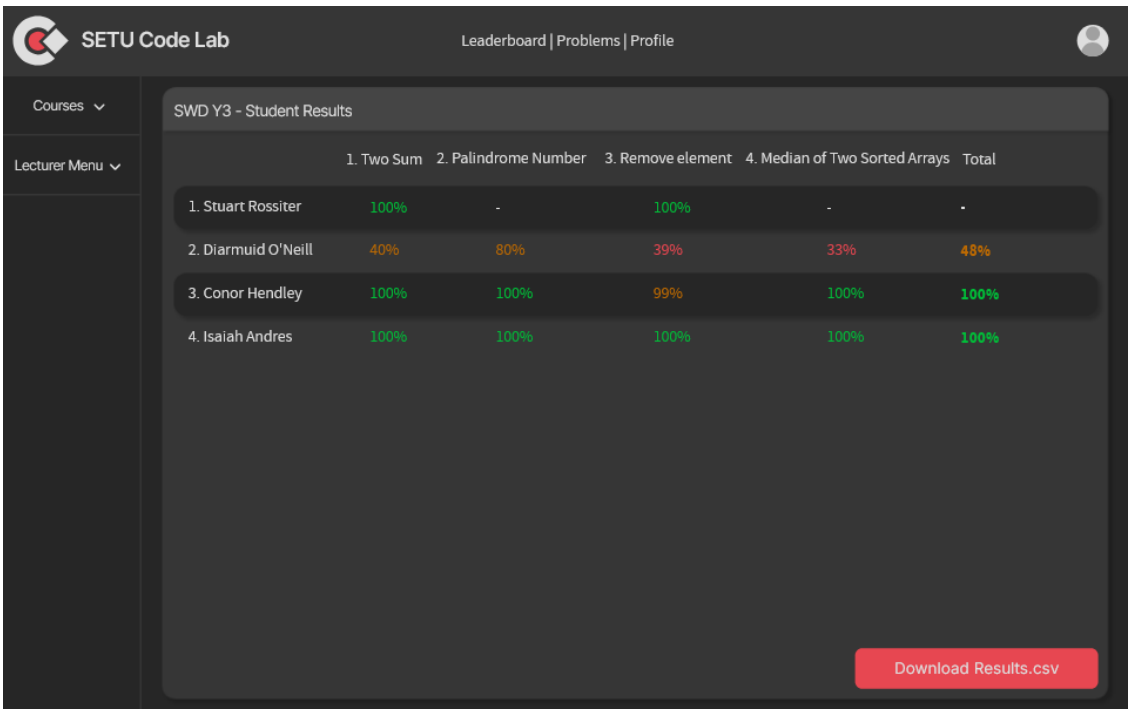

Figure 29. *View Course screen design*

View Problem > Manage Course > View Course > View Result

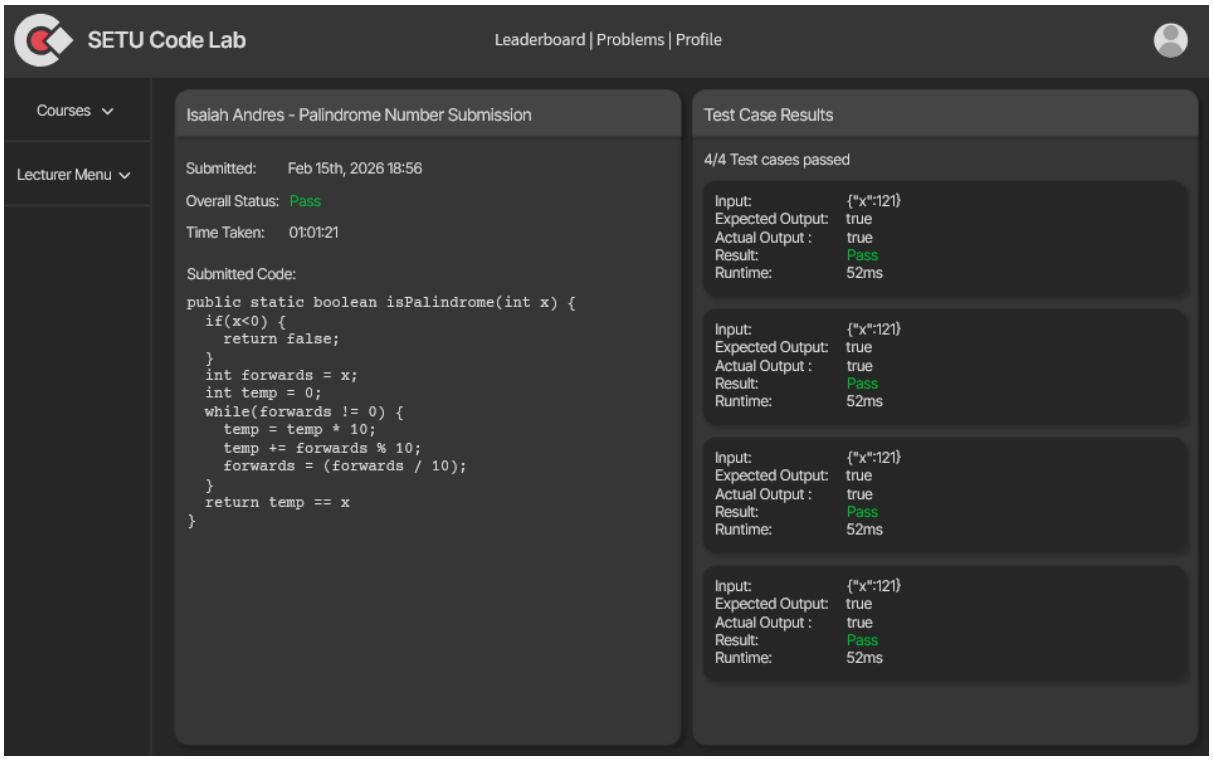

Figure 30. *View Result Screen design*

View Problem > Profile

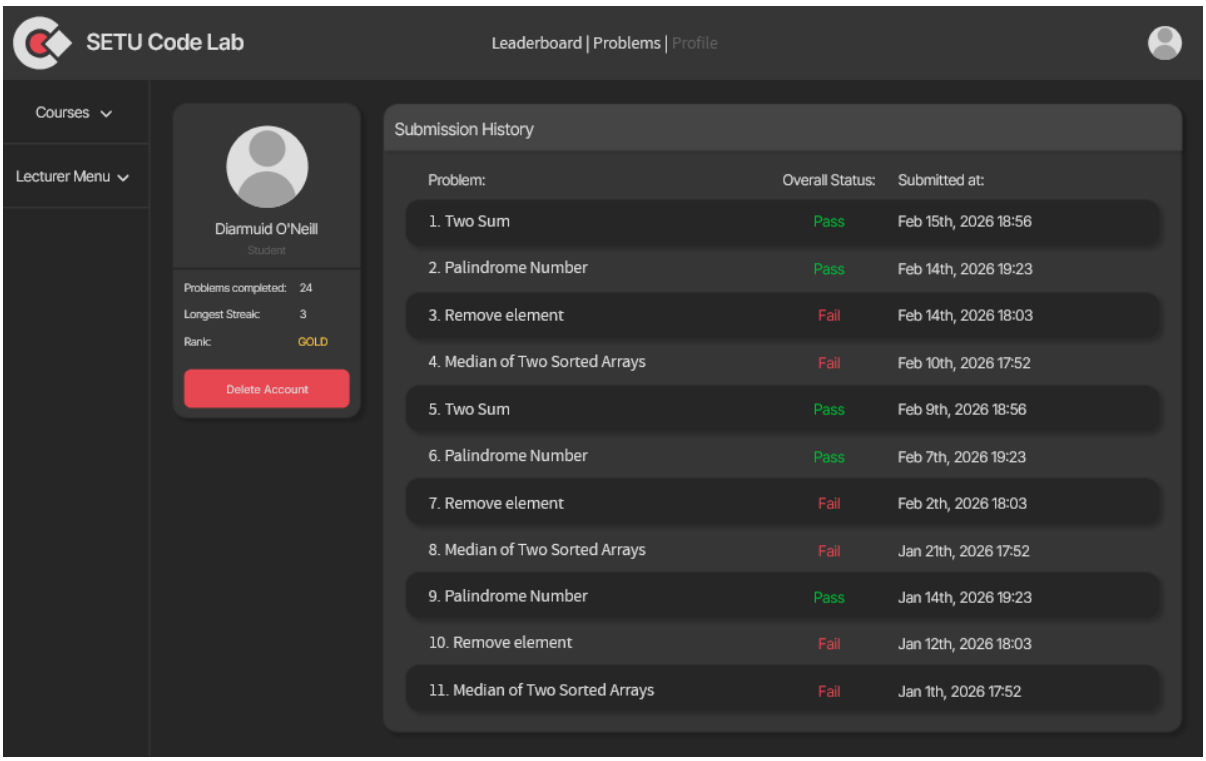

Figure 31. *View Profile screen design*

View Problem > Leaderboard

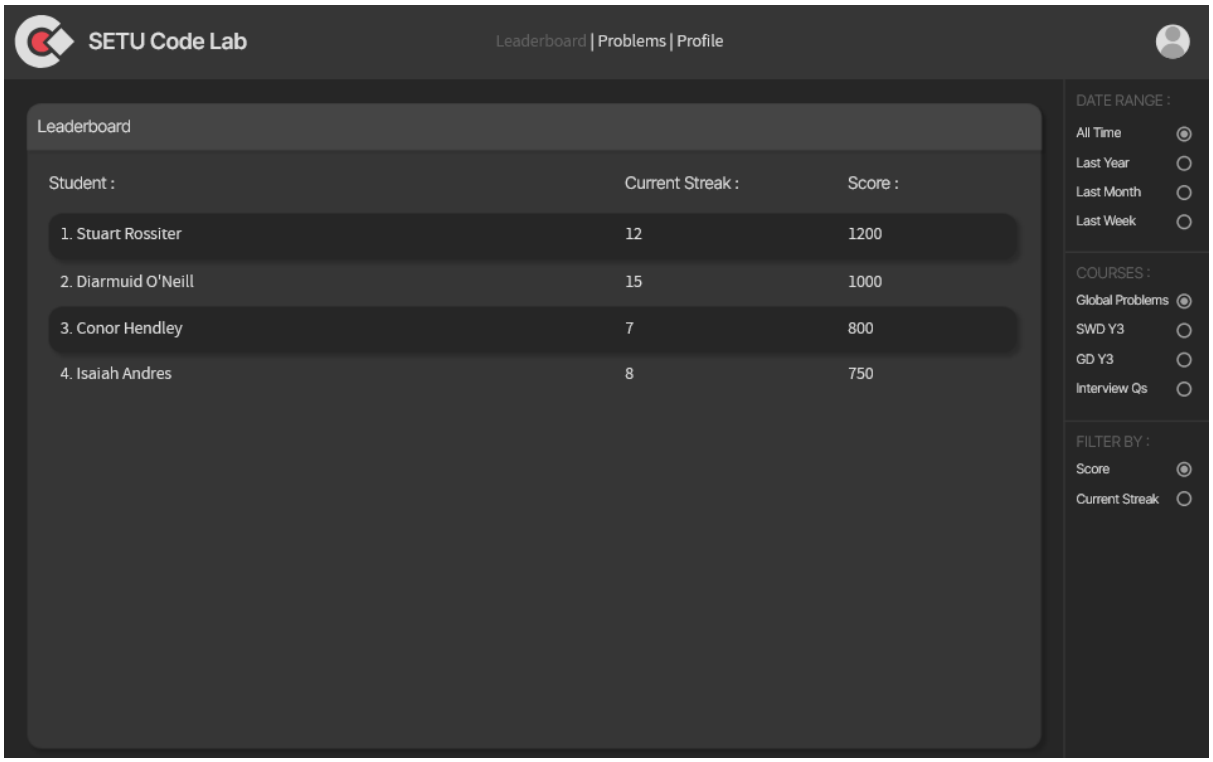

Figure 32. *Leaderboard screen design*

Bibliography

CodeMirror, 2026. *CodeMirror - System Guide*. [Online]
Available at: https://codemirror.net/docs/guide/
[Accessed 5 March 2026].

Dias, P., 2025. *NPM - dockerode*. [Online]
Available at: https://www.npmjs.com/package/dockerode
[Accessed 5 March 2026].

Haverbeke, M., 2025. *NPM - @codemirror/lang-java*. [Online]
Available at: https://www.npmjs.com/package/@codemirror/lang-java
[Accessed 5 March 2026].

Haverbeke, M., 2026. *Github - codemirror lang/python*. [Online]
Available at: https://github.com/codemirror/lang-python
[Accessed 17 March 2026].

Hu, D., 2026. *React Spinners*. [Online]
Available at: https://www.davidhu.io/react-spinners/
[Accessed 5 March 2026].

John Otander, T. W. R. H., 2025. *NPM - react-markdown*. [Online]
Available at: https://www.npmjs.com/package/react-markdown
[Accessed 5 March 2026].

Labib, A., 2025. *NPM - react-timer-hook*. [Online]
Available at: https://www.npmjs.com/package/react-timer-hook
[Accessed 5 March 2026].

Marijn Haverbeke, e. a., 2026. *NPM - @codemirror/language*. [Online]
Available at: https://www.npmjs.com/package/@codemirror/language
[Accessed 5 March 2026].

S., R., 2025. *NPM - cookie-parser*. [Online]
Available at: https://www.npmjs.com/package/cookie-parser
[Accessed 5 March 2026].

TypeScript, 2026. *NPM - @types/dockerode*. [Online]
Available at: https://www.npmjs.com/package/@types/dockerode
[Accessed 5 March 2026].

Appendix

The following CREATE TABLE statements define the full database schema used in this project.

users

```sql
CREATE TABLE users (
    user_id          SERIAL PRIMARY KEY,
    user_name        TEXT NOT NULL,
    email            VARCHAR(255) UNIQUE NOT NULL,
    password         VARCHAR(255) NOT NULL,
    role             VARCHAR(50) NOT NULL CHECK (role IN ('student', 'lecturer')),
    total_points     INTEGER NOT NULL DEFAULT 0,
    current_streak   INTEGER NOT NULL DEFAULT 0,
    longest_streak   INTEGER NOT NULL DEFAULT 0,
    last_solved_date DATE
);
```

problem

```sql
CREATE TABLE problem (
    problem_id          SERIAL PRIMARY KEY,
    user_id             INTEGER REFERENCES users(user_id),
    problem_title       VARCHAR(255),
    problem_description TEXT,
    difficulty          INTEGER CHECK (difficulty BETWEEN 1 AND 5),
    placeholder_code    TEXT,
    points              INTEGER NOT NULL DEFAULT 0
);
```

test_case

```
CREATE TABLE test_case (
    test_case_id   SERIAL PRIMARY KEY,
    problem_id     INT REFERENCES problem(problem_id) ON DELETE CASCADE,
    input_value    JSONB NOT NULL,
    expected_value JSONB NOT NULL
);
```

submission

```
CREATE TABLE submission (
    submission_id  SERIAL PRIMARY KEY,
    user_id        INT REFERENCES users(user_id),
    problem_id     INT REFERENCES problem(problem_id) ON DELETE CASCADE,
    submitted_code TEXT,
    submitted_at   TIMESTAMP DEFAULT now(),
    overall_status BOOLEAN,
    time_taken     INT,
    percentage     INT,
    points_awarded INTEGER NOT NULL DEFAULT 0
);
```

test_case_result

```
CREATE TABLE test_case_result (
    test_case_result_id SERIAL PRIMARY KEY,
    submission_id       INT REFERENCES submission(submission_id) ON DELETE
CASCADE,
    test_case_id        INT REFERENCES test_case(test_case_id) ON DELETE CASCADE,
    passed              BOOLEAN,
    actual_output       TEXT,
    runtime_ms          INTEGER
);
```

course

```sql
CREATE TABLE course (
    course_id          SERIAL PRIMARY KEY,
    owner_id           INT REFERENCES users(user_id) ON DELETE CASCADE,
    course_title       VARCHAR(255),
    course_description TEXT,
    created_at         TIMESTAMP DEFAULT now()
);
```

enrollment

```sql
CREATE TABLE enrollment (
    course_id INT REFERENCES course(course_id) ON DELETE CASCADE,
    user_id   INT REFERENCES users(user_id) ON DELETE CASCADE,
    PRIMARY KEY (course_id, user_id)
);
```

course_problem

```sql
CREATE TABLE course_problem (
    course_id  INT REFERENCES course(course_id) ON DELETE CASCADE,
    problem_id INT REFERENCES problem(problem_id) ON DELETE CASCADE,
    PRIMARY KEY (course_id, problem_id)
);
```